



Build-Measure-Learn Cycle Planning

Rahbar “AI-Powered Tax Filing Assistant”

Team Members:

Muhammad Haris (2023428)

Mumtaz Ali (2023559)

Ahmad Ibrahim Ahmer (2023066)

Course: CS-391

Batch: 33

Section: BSCS

Instructor: Dr.Fahd.S

MVP CREDENTIALS:

Sign-in details:

Email: harris.giki@gmail.com

Password: userpassword123

MVP Link: <https://rahbar.vercel.app/>

PART A: Actionable Metrics for the Measure Phase

Quantitative Metrics

We selected three measurable, behavior-driven metrics built directly into our MVP. Each one is intentionally designed to inform product decisions, validate assumptions, and reveal friction points rather than simply track surface-level usage.

Metric 1: Task Completion Rate (TCR)

Definition:

The proportion of users who are able to move from the beginning of the automated tax-filing experience all the way to a successfully prepared submission.

How We Measure It:

- Every automation step (authentication, navigation, form completion) is logged in MongoDB with precise timestamps.
- At the end of each run, the attempt is classified as completed, abandoned, or failed.

Formula:

$TCR = (\text{Successful Completions} \div \text{All Automation Attempts}) \times 100$

Target Goal: At least 85% completion.

What It Tells Us:

A declining TCR may point to:

- Outdated or unstable web automation selectors
- Confusing or poorly structured dialogue in the voice interaction layer
- Recent changes to the FBR IRIS portal that the automation hasn't adapted to

Decision Rule:

If TCR drops below 70%, we will analyze automation logs, update selectors, and conduct A/B testing on conversation flows to pinpoint and resolve failure sources.

Metric 2: Voice Session Duration to Automation Trigger (VSDAT)

Definition:

The average time between the start of a user's voice session and the moment the automation pipeline successfully initiates.

How We Measure It:

- The session begins when the LiveKit connection is established.
- The moment a data channel event triggers automation is recorded.
- These timestamps are compared and stored with session metadata.

Formula:

$$\text{VSDAT} = \Sigma(\text{Time to Automation Trigger}) \div \text{Number of Sessions}$$

Target Goal:

- ≤ 3 minutes for sessions where user data is pre-filled
- ≤ 8 minutes for sessions requiring full information collection

What It Tells Us:

- Longer sessions may indicate unclear prompts or inefficient dialogue design.
- Very short sessions followed by failure suggest premature triggers.
- Comparing Urdu-only and bilingual interactions helps uncover comprehension issues.

Decision Rule:

If VSDAT exceeds 10 minutes during full data collection, we will streamline prompts and reduce the number of required details before automation begins.

Metric 3: Error Recovery Rate (ERR)

Definition:

The percentage of automation-related errors that are resolved successfully within the same user session, either automatically or through user-driven retries.

How We Measure It:

- Every error event is logged with its category (timeouts, selector issues, authentication failures, etc.).
- Each retry attempt is tracked and matched with success or failure outcomes.
- Re-initiation events by users are connected to earlier errors.

Formula:

$ERR = (Recovered\ Errors \div Total\ Errors) \times 100$

Target Goal: At least 60% successful recovery.

What It Tells Us:

- Low ERR for a particular category signals where deeper technical fixes are needed.
- High recovery rates demonstrate robust fallback behavior in the automation system.
- Error patterns help differentiate systemic weaknesses from isolated anomalies.

Decision Rule:

If any category falls below 30% recovery, we will implement tailored handling logic or build user prompts to guide manual resolution.

Qualitative Metrics: Customer Discovery Interview Plan

Interview Objective

To obtain rich, experience-based feedback from users who have engaged with Rahbar, uncovering insights into usability, trust, clarity, and any unmet expectations that arise during real use.

Target Interview Participants (Minimum 10)

User Segment	Count	Selection Criteria
First-time Tax Filers	2	No independent tax filing experience
Repeat Manual Filers	4	Filed for 2+ years without automation

Tech-Comfortable Users	2	Frequent users of digital finance tools
Tech-Hesitant Users	2	Limited digital literacy; prefers offline processes

Interview Format (At-most 20 Minutes)

1. Pre-Interaction Context

- Past tax filing methods
- Awareness and perception of the IRIS portal
- Key challenges in current filing processes

2. Voice Agent Experience

- First impression of the Urdu voice interface
- Clarity of system questions
- Comfort level with sharing financial data with AI
- Voice versus text preference

3. Observing Automation

- Reaction to watching the automated browser actions
- Interpretation of progress indicators
- Feelings of trust, uncertainty, or reassurance
- Perception of speed and accuracy

4. Overall Outcome

- Satisfaction with the finishing output
- Likelihood of returning next year
- Recommendation potential (NPS)
- Desired improvements

5. Open Feedback

- Surprises, positive or negative

- Missing expectations
- Budget sensitivity toward premium add-ons

Analysis Approach

- Thematic coding to identify repeated patterns
- Sentiment breakdown per user segment
- Prioritization of feature requests
- Mapping of trust-related concerns to product decisions

PART B: Technical Depth and Complexity Statement

Core Complex Component: Real-Time Voice-to-Automation Orchestration

Overview

The most technically advanced part of Rahbar is its orchestration system, which tightly integrates real-time voice understanding, distributed state management, and resilient browser automation. It ensures that natural conversation flows smoothly into a precise, fully automated filing process while operating across three different environments.

Technical Architecture

The system spans three coordinated platforms:

1. Voice AI Layer (LiveKit Cloud):
A Python-driven agent built with the LiveKit Agents SDK, using Deepgram, ElevenLabs, and GPT-4 to support natural Urdu dialogue.
2. Frontend Interface (Vercel Edge):
A Next.js application managing authentication, session lifecycle, and real-time visual updates.
3. Automation Backend (Railway Docker):
A FastAPI service using Playwright to interact with the IRIS portal through headless Chromium.

Advanced Computing Concepts Utilized

1. Distributed State Machine with Eventual Consistency

The user journey spans multiple systems, each holding a partial state. We synchronize these states over encrypted WebRTC data channels with minimal latency, using logical timestamps and last-writer-wins rules to avoid conflicts. This ensures that voice interactions, interface updates, and automation steps remain aligned in real time.

2. Multi-Strategy Resilient Selector System

Because the IRIS portal frequently changes layout and offers few stable identifiers, selectors rely on layered fallback strategies (IDs, CSS attributes, XPath, text-based queries, and coordinate approximations). The automation engine also incorporates adaptive wait strategies and simple visual regression checks to detect unexpected page shifts.

3. Multilingual Natural Language Understanding for Urdu

Urdu poses structural and linguistic challenges, especially with code-switching. The system combines GPT-4’s conversational reasoning with rule-based extraction for structured fields. This hybrid approach handles numeric variations, domain-specific terminology, and references across multiple conversation turns.

4. Secure Separation of Credentials

Sensitive user data is safeguarded by encryption-at-rest in MongoDB Atlas, isolation within container memory, masked automation logs, and single-use session tokens.

Why This Complexity Was Necessary:

The Core Problem

We needed to enable users many with limited digital familiarity—to file taxes confidently through conversational Urdu, all while automating interactions with a portal that offers no API and changes unpredictably.

Conflicting Constraints and Resolutions

Constraint

Conflict

Solution

Real-time UX vs. reliable automation	Fast updates can disrupt strict automation sequencing	WebRTC for fast sync, synchronous waits for critical actions
Conversational freedom vs. strict form requirements	Voice input is open-ended; FBR fields are rigid	Hybrid GPT-4 + rule-based extraction
Transparency vs. privacy	Users want visibility but must have protected data	Real-time screenshots with masked fields
Unstable portal vs. reliable completion	Layout changes break scripts	Multi-layer selector fallbacks

Why Simpler Approaches Don't Work:

- Browser extensions cannot handle multi-step, conditional workflows.
- Chat-only systems exclude users who struggle with typing.
- A monolithic deployment cannot scale voice, automation, and UI requirements efficiently.
- Direct integration is impossible due to the lack of official IRIS APIs.

Technical Innovation

Rahbar introduces a Voice-Triggered RPA (VT-RPA) model that blends real-time AI conversation with adaptive browser automation an approach especially suited for countries with unstable government portals and diverse user technical backgrounds.

Conclusion

Rahbar's complexity lies in harmonizing advanced voice AI, distributed synchronization, adaptive automation, and stringent security into a single seamless experience. The user only speaks; the system orchestrates every other layer beneath that simplicity.

Appendix: Technology Stack Overview

Layer	Technologies	Purpose
Voice AI	LiveKit Agents, Deepgram, ElevenLabs, GPT-4	Urdu interaction and understanding

Frontend	Next.js 16, React, TailwindCSS, LiveKit Client	User experience and live updates
Backend	FastAPI, Playwright, asyncio	Robust automation and orchestration
Database	MongoDB Atlas	Secure data and session storage
Hosting	Vercel, Railway, LiveKit Cloud	Distributed deployment
Communication	WebRTC data channels	Real-time and reliable messaging